

МИНЕСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
бюджетное учреждение высшего образования
«Московский технический университет связи и информатики»

Отчёт по лабораторной работе №8
по дисциплине
«Информационные технологии и программирование»

Выполнил:
студент группы БПИ2402
Лукьянов Иван Денисович

Москва, 2025 г.
Цель работы:

Освоить основные приёмы многопоточного программирования в Java, включая использование интерфейса Callable, ExecutorService и Future для реализации параллельных вычислений, а также применить синхронизацию для безопасного доступа к общим ресурсам.

Выполнение задания:

1. Аннотация @DataProcessor

Пользовательская аннотация, помечается методами, которые должны быть выполнены при обработке данных. Аннотация сохраняется на этапе выполнения программы (RUNTIME) и может применяться к методам (METHOD).

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface DataProcessor {
}
```

Это позволяет использовать рефлексию для поиска и вызова методов в процессе выполнения.

```
1  import java.lang.annotation.Retention;
2  import java.lang.annotation.RetentionPolicy;
3  import java.lang.annotation.Target;
4  import java.lang.annotation.ElementType;
5
6  @Retention(RetentionPolicy.RUNTIME)
7  @Target(ElementType.METHOD)
8  public @interface DataProcessor {
9  }
10
```

2. Класс Processor с методами обработки данных

Класс содержит два метода, помеченных аннотацией @DataProcessor:

- `filterAdults(List<String> input)` — использует Stream API для фильтрации строк, выделяет взрослых (возраст ≥ 18 лет). Данные парятся как пары "имя,возраст", фильтруются по возрасту и собираются обратно в список;
- `uppercaseNames(List<String> input)` — использует Stream API для трансформации: преобразует имена в верхний регистр, сохраняя информацию о возрасте в формате "ИМЯ,возраст".

Оба метода демонстрируют различные операции Stream API: фильтрацию (`filter`), трансформацию (`map`) и терминальную операцию (`collect`).

```

1  import java.util.List;
2  import java.util.stream.Collectors;
3
4  public class Processor {
5
6      @DataProcessor
7      public List<String> filterAdults(List<String> input) {
8          return input.stream()
9              .filter(line -> {
10                  String[] parts = line.split(",");
11                  int age = Integer.parseInt(parts[1]);
12                  return age >= 18;
13              })
14              .collect(Collectors.toList());
15      }
16
17      @DataProcessor
18      public List<String> uppercaseNames(List<String> input) {
19          return input.stream()
20              .map(line -> {
21                  String[] parts = line.split(",");
22                  String name = parts[0].toUpperCase();
23                  return name + "," + parts[1];
24              })
25              .collect(Collectors.toList());
26      }
27  }

```

3. Класс DataManager — управление данными и параллельная обработка

DataManager отвечает за:

- **Регистрацию обработчиков** — метод `registerProcessor(Object processor)` добавляет объекты, содержащие методы с аннотацией `@DataProcessor`;
- **Загрузку данных** — метод `loadData(String filePath)` читает строки из файла с использованием `Files.readAllLines()`;
- **Динамический поиск методов** — метод `processData()` использует рефлексия для поиска всех методов с аннотацией `@DataProcessor` и для каждого создаёт задачу;
- **Параллельное выполнение** — каждый найденный метод оборачивается в `Callable<List<String>>` и отправляется в пул потоков `ExecutorService` (размер = количество доступных процессоров);

- **Сбор результатов** — результаты из Future объектов собираются в единый список обработанных данных;
- **Сохранение результатов** — метод saveData(String filePath) записывает обработанные данные в новый файл с помощью Files.write()).

```
1  import java.lang.reflect.Method;
2  import java.nio.file.Files;
3  import java.nio.file.Paths;
4  import java.io.IOException;
5  import java.util.ArrayList;
6  import java.util.List;
7  import java.util.concurrent.*;
8
9  public class DataManager {
10
11      private final List<Object> processors = new ArrayList<>();
12
13      private List<String> rawData = new ArrayList<>();
14      private List<String> processedData = new ArrayList<>();
15
16      private final ExecutorService executor =
17          Executors.newFixedThreadPool(Runtime.getRuntime().availableProcessors());
18
19      public void registerProcessor(Object processor) {
20          processors.add(processor);
21      }
22
23      public void loadData(String filePath) throws IOException {
24          rawData = Files.readAllLines(Paths.get(filePath));
25          System.out.println("Загружены данные: " + rawData);
26      }
27
28      public void processData() throws Exception {
29          if (rawData.isEmpty()) {
30              System.out.println("Нет данных для обработки.");
31              return;
32          }
33      }
```

```

34     List<Future<List<String>>> futures = new ArrayList<>();
35
36     for (Object processor : processors) {
37         for (Method method : processor.getClass().getDeclaredMethods()) {
38             if (method.isAnnotationPresent(DataProcessor.class)) {
39
40                 Callable<List<String>> task = () -> {
41                     @SuppressWarnings("unchecked")
42                     List<String> result =
43                         (List<String>) method.invoke(processor, rawData);
44                     System.out.println("Метод " + method.getName()
45                                         + " выполнен в потоке "
46                                         + Thread.currentThread().getName());
47                     return result;
48                 };
49
50                 futures.add(executor.submit(task));
51             }
52         }
53     }
54
55     processedData = new ArrayList<>();
56     for (Future<List<String>> f : futures) {
57         processedData.addAll(f.get());
58     }
59
60     System.out.println("Итоговые данные: " + processedData);
61 }
62
63 public void saveData(String filePath) throws IOException {
64     Files.write(Paths.get(filePath), processedData);
65     System.out.println("Результат сохранён в " + filePath);
66 }
67
68 public void shutdown() {
69     executor.shutdown();
70 }
71 }

```

4. Класс Main

```
1 public class Main {
2
3     Run | Debug
4     public static void main(String[] args) throws Exception {
5
6         DataManager manager = new DataManager();
7
8         manager.registerProcessor(new Processor());
9
10        manager.loadData(filePath: "file.txt");
11
12        manager.processData();
13
14        manager.saveData(filePath: "result.txt");
15
16        manager.shutdown();
17    }
```

Исходные данные и результат:

1	Naruto,17	1	Kakashi,30
2	sasuke,17	2	Itachi,21
3	sakura,16	3	jiraiya,50
4	Kakashi,30	4	Luffy,19
5	Itachi,21	5	Zoro,21
6	Hinata,16	6	nami,18
7	jiraiya,50	7	Sanji,21
8	Luffy,19	8	Robin,28
9	Zoro,21	9	Levi,30
10	nami,18	10	Goku,24
11	Sanji,21	11	Vegeta,29
12	Robin,28	12	hisoka,28
13	Levi,30	13	NARUTO,17
14	Eren,15	14	SASUKE,17
15	Mikasa,15	15	SAKURA,16
16	Goku,24	16	KAKASHI,30
17	Vegeta,29	17	ITACHI,21
18	Gon,12	18	HINATA,16
19	Killua,12	19	JIRAIYA,50
20	hisoka,28	20	LUFFY,19
21		21	ZORO,21
		22	NAMI,18
		23	SANJI,21
		24	ROBIN,28
		25	LEVI,30
		26	EREN,15
		27	MIKASA,15
		28	GOKU,24
		29	VEGETA,29
		30	GON,12
		31	KILLUA,12
		32	HISOKA,28

Вывод:

В ходе выполнения лабораторной работы №8 были освоены ключевые концепции современной Java:

- **Аннотации** позволяют добавлять метаданные к коду и использовать их для конфигурации поведения приложения;
- **Stream API** предоставляет функциональный подход к обработке коллекций, делая код более выразительным и компактным;
- **ExecutorService** обеспечивает удобное управление многопоточными вычислениями и параллельной обработкой данных;
- **Рефлексия** позволяет разрабатывать гибкие и расширяемые приложения, которые могут динамически адаптироваться к наличию определённых методов и аннотаций.

Ссылка на мой Github репозиторий с лабораторной:

<https://github.com/Furibado/Java-lab-work>